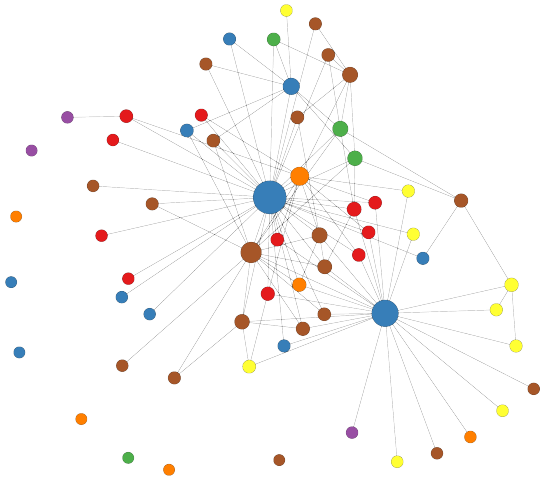
Title of Presentation

Topics



March, 2026

1. Components
2. Examples

1 Components
Lists
Maths
Tables
Figures
Others

2 Examples

1 Components
Lists
Maths
Tables
Figures
Others

2 Examples

Simple list

- Google
- Microsoft
- Apple
- Amazon
- Meta

Enumerate list

1. Python
2. Java
3. C++
4. C
5. Go

List of list

- Install requirements
  - `Python`
  - `Numpy`
  - `Pandas`
  - `NetworkX`
  - `Matplotlib`
- Topics
  - Create graphs
  - Graphs visualization
- Links
  - Open in GitHub
  - Open in Google Colab

Multicols list

- Computer vision
- Natural language processing
- Recommender systems
- Program analysis
- Knowledge graphs
- Bioinformatics
- Chemistry
- Biology
- Physic
- Anomaly detection
- Urban intelligence
- Financial transaction

Description list

Input: Graph data

Output: Community labels

Method: Leiden algorithm

Sequential list display

- Many people use LaTeX, and many universities have their own Beamer themes

Sequential list display

- Many people use LaTeX, and many universities have their own Beamer themes
- For Chinese support, use XeLaTeX as the compiler

List (dynamic) I

- This template is modified from a Tsinghua University project
- Original template: `https://www.overleaf.com/latex/templates/thu-beamer-theme/vwnqmzndvwyb`
- Minor modifications were made
- Style adjusted to personal preference
- Added small Beamer usage examples
- Replaced default fonts with more readable ones
- Fixed inconsistent Chinese font issues
- Changed math fonts to serif for better appearance

List (dynamic) II

- **Use `\alert` to emphasize text**
- This template is modified from a Tsinghua University project
- Original template: `https://www.overleaf.com/latex/templates/thu-beamer-theme/vwnqmzndvwyb`

1 Components
Lists
Maths
Tables
Figures
Others

2 Examples

Simple equations

$V = \frac{4}{3}\pi r^3$

$$V = \frac{4}{3}\pi r^3$$

Equations

$$E = mc^2$$

$$J(\theta) = \mathbb{E}_{\pi_\theta}[G_t] \tag{1}$$

$$Q_{\text{target}} = r + \gamma Q^{\pi}(s', \pi_\theta(s') + \varepsilon) \tag{2}$$

$$\varepsilon \sim \text{clip}(\mathcal{N}(0, \sigma), -c, c) \tag{3}$$

Equation with numbers

$$
\begin{aligned}
A = \lim_{n\to\infty} \Delta x \Big(a^2 + \Big(a^2 + 2a\Delta x + (\Delta x)^2 \Big) \\
+ \Big(a^2 + 2\cdot 2a\Delta x + 2^2 (\Delta x)^2 \Big) \\
+ \Big(a^2 + 2\cdot 3a\Delta x + 3^2 (\Delta x)^2 \Big) \\
+ \dots \\
+ \Big(a^2 + 2\cdot (n-1)a\Delta x + (n-1)^2 (\Delta x)^2 \Big) \Big) \\
= \frac{1}{3}\left(b^3 - a^3\right) \quad (4)
\end{aligned}
$$

Algorithms

Algorithm 1: My algorithm.

Input : G, s ▷ Graph G and a source node s

Output: All nodes reachable from s

```
Q ← ∅                                  ▷ Create an empty queue
V ← ∅                                  ▷ Create an empty set of visited
enqueue(Q, s)
append(V, s)
while Q ≠ ∅ do
    v ← dequeue(Q)
    print(v)
    foreach u ∈ N(v) do
        if u ∉ V then
            append(V, u)
            enqueue(Q, u)
```

1 Components
Lists
Maths
Tables
Figures
Others

2 Examples

Table

Table 1: My table.

Representation	Complexity
Adjacency matrix	$O(\|V\|^2)$
Adjacency list	$O(\|V\| + \|E\|)$
Edge list	$O(\|E\|)$
Incidence matrix	$O(\|V\|\|E\|)$

Table (custom)

Table 2: My table.

Representation	Advantages	Disadvantages
Adjacency matrix	Simple structure, constant-time edge lookup.	High memory usage for large sparse graphs.
Adjacency list	Memory efficient for sparse graphs, easy to iterate over neighbors.	Edge existence check may be slower.
Edge list	Very simple representation, useful for algorithms that process edges sequentially.	Inefficient for neighbor queries.
Incidence matrix	Useful in some theoretical formulations and algebraic graph analysis.	High memory usage and rarely used in practical implementations.

Table (custom)

Table 3: My table.

IA	Tasks
AI learns patterns	AI automates tasks
AI processes data	AI supports decisions
AI powers assistants	AI transforms industries
AI evolves	AI predicts

1 Components
Lists
Maths
Tables
Figures
Others

2 Examples

Figure

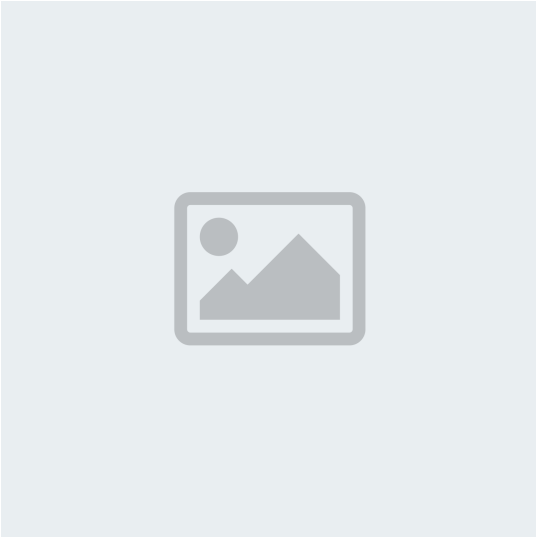


Figure 1: My image.
Source: Author, 2024.

Multiple figures

(a) My image. (b) My image. (c) My image. (d) My image.

Figure 2: My image.
Source: Author, 2024.

Figure (dynamic)

Figure 3: My image.
Source: Author, 2024.

1 Components
Lists
Maths
Tables
Figures
Others

2 Examples

Blocks

Info

General content

Warning

Important message

Example

Example content

Frame (two columns)

- **Graph traversal**
  - Breadth-First Search
  - Depth-First Search
- **Shortest path algorithms**
  - Dijkstra
  - FloydWarshall
  - Bellman-Ford

1 Components

2 Examples

Example 1

In mathematics and computer science, a graph (network) is defined as $G = (V, E)$, where $V = \{v_1, v_2, ..., v_n\}$ is the nodes (or *vertices*, or *points*) set and $E = \{e_1, e_2, ..., e_m\}, e_i \in V \times V$ a set of edges (or *lines*, or *arcs*) formed by unordered pairs of nodes.

Number of nodes is denoted $|V|$.
Number of edges is denoted $|E|$.

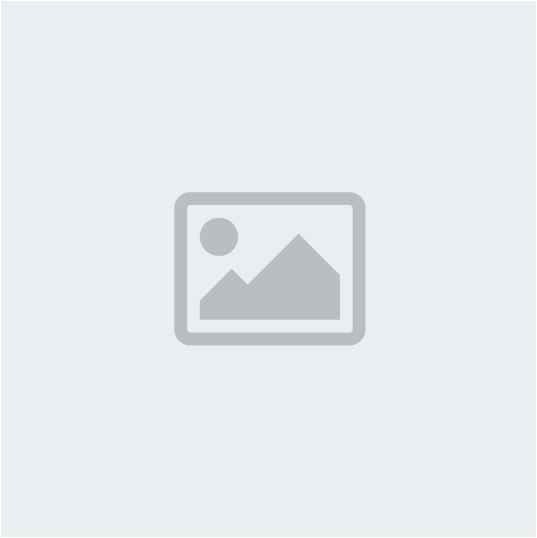


Figure 4: My image.
Source: Author, 2024.

Example 2

Furthermore, each edge $e \in E$ is said to join two vertices. If e joins $u, v \in V$, then $e = (u, v)$. Vertices u and v are said to be adjacent.

Neighborhood

$$N(v) = \{w \in V v \neq w \wedge (v, w) \in E\} \tag{5}$$

Example 3

Adjacency matrix

Adjacency matrix A, is an $n \times n$ matrix where:

$$A_{i,j} = \begin{cases} 1 & \text{if there is an edge from vertex } i \text{ to vertex } j \\ 0 & \text{otherwise} \end{cases}$$

(a) My image.

(b) My image.

Figure 5: My image.
Source: Author, 2024.

Example 4

Idea

Does the shortest path from a to c go through b?

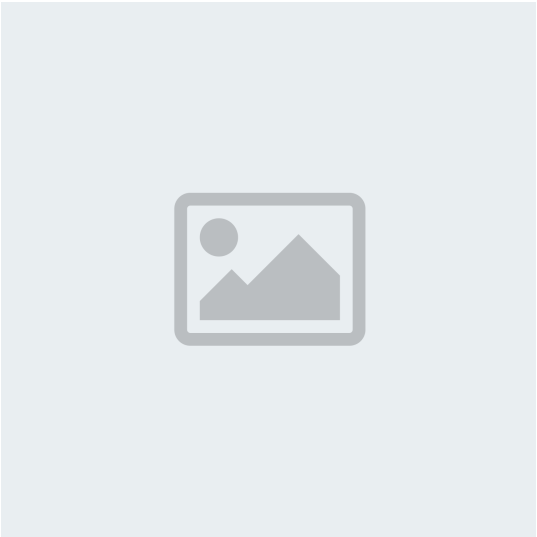


Figure 6: My image.
Source: Author, 2024.

Example 5

BFS is a graph traversal algorithm that explores nodes level by level, visiting all nodes at a given distance from the starting node before moving to the next level.

Description:

- Uses a queue (FIFO) to manage the nodes to be explored.
- Marks visited nodes to avoid cycles or repeated visits.
- Ideal for finding the shortest path in unweighted graphs.

Example 6

In this lab, students will apply basic programming concepts to solve problems, develop coding skills, and practice writing and debugging efficient solutions.

- Requirements
  - `Python`
  - `Numpy`
  - `Pandas`
  - `NetworkX`
  - `Matplotlib`
- Topics
  - Create graphs
  - Graphs visualization
- Lab
  - Open in Google Colab

Example 7

This section introduces key equations to model and analyze the problem.

Main equation

$$J(\theta) = \mathbb{E}_{\pi_\theta}[G_t]$$

Supplementary equations

$$Q_{\text{target}} = r + \gamma Q^{\pi}(s', \pi_\theta(s') + \varepsilon) \tag{6}$$

$$\varepsilon \sim \text{clip}(\mathcal{N}(0, \sigma), -c, c) \tag{7}$$

Solution

$$
\begin{aligned}
A = \lim_{n\to\infty} \Delta x \Big(a^2 + \Big(a^2 + 2a\Delta x + (\Delta x)^2 \Big) \\
+ \Big(a^2 + 2\cdot 2a\Delta x + 2^2 (\Delta x)^2 \Big) \\
+ \Big(a^2 + 2\cdot 3a\Delta x + 3^2 (\Delta x)^2 \Big) \\
+ \dots \\
+ \Big(a^2 + 2\cdot (n-1)a\Delta x + (n-1)^2 (\Delta x)^2 \Big) \Big) \\
= \frac{1}{3}\left(b^3 - a^3\right) \quad (8)
\end{aligned}
$$

Example 8

1. Visit the adjacent unvisited node. Mark as visited, display, and insert (queue) in a queue.
2. In case no adjacent node is found, remove the first node from the queue.
3. Repeat steps 1, and 2 until the queue is empty.

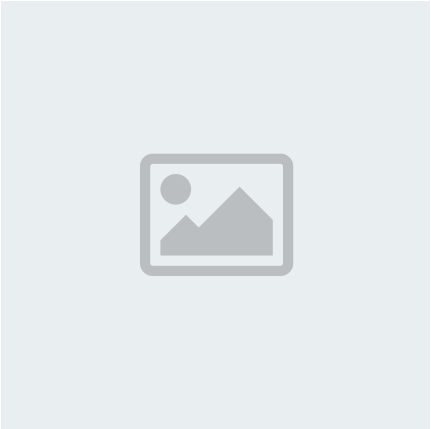


Figure 7: My image.
Source: Author, 2024.

Example 9

The following table summarizes the key information and results, providing a clear overview of the main variables and their relationships.

Table 4: My table.

Representation	Complexity
Adjacency matrix	$O(\|V\|^2)$
Adjacency list	$O(\|V\| + \|E\|)$
Edge list	$O(\|E\|)$
Incidence matrix	$O(\|V\|\|E\|)$

Example 10

The following table summarizes the key information and results, providing a clear overview of the main variables and their relationships.

Table 5: My table.

Representation	Complexity
Adjacency matrix	$O(\|V\|^2)$
Adjacency list	$O(\|V\| + \|E\|)$
Edge list	$O(\|E\|)$
Incidence matrix	$O(\|V\|\|E\|)$

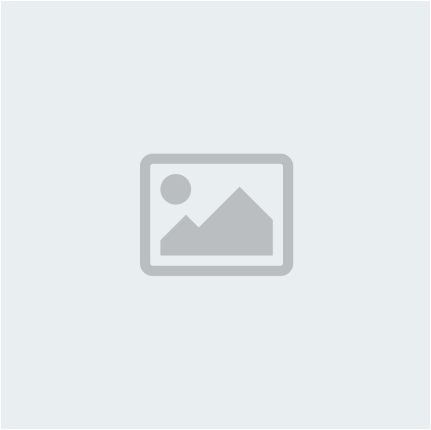


Figure 8: My image.
Source: Author, 2024.

Example 11

The following table summarizes the key information and results, providing a clear overview of the main variables and their relationships.

1. **Análisis:** Definir requisitos y comprender el problema.
2. **Diseño:** Planificar la estructura del algoritmo.
3. **Implementación:** Escribir el código en un lenguaje de programación.
4. **Prueba:** Verificar el funcionamiento y corregir errores.

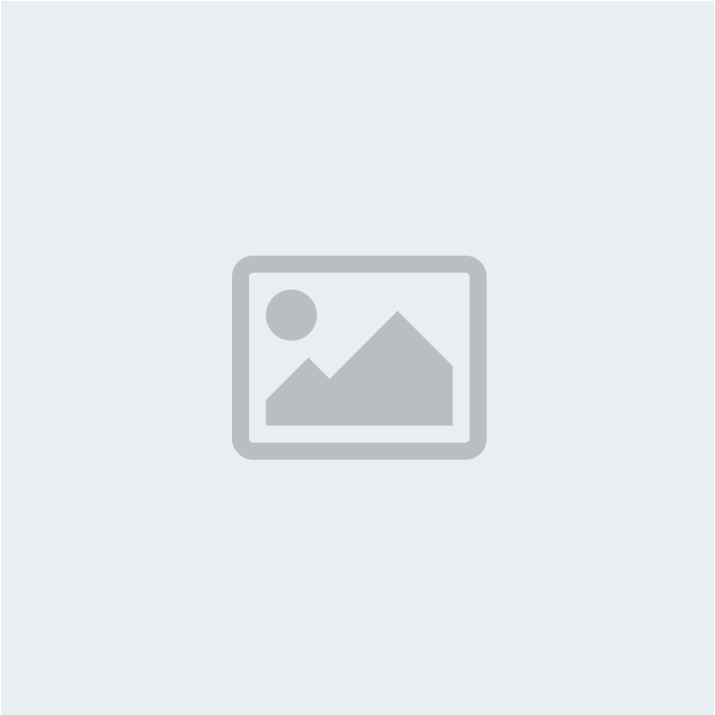


Figure 9: My image.
Source: Author, 2024.

Thanks!

∞

https://github.com/win7

- Erciyes, K. (2018). *Guide to graph algorithms*
- Yao Ma, Jiliang Tang. (2021). *Deep Learning on Graphs*.
- Stamile C., Marzullo A., Deusebio E. (2021). *Graph Machine Learning*.
- Wu, L., Cui, P., Pei, J., Zhao, L., & Guo, X. (2022). *Graph neural networks: foundation, frontiers and applications*.
- Knickerbocker, D. (2023). *Network Science with Python*.
- Labonne, M. (2023). *Hands-On Graph Neural Networks Using Python*.
- Broadwater, K., & Stillman, N. (2025). *Graph neural networks in action*.